LibanPost eShop — feature inventory

Contents

Contents . . . 2
Platform and architecture . . . 2
Storefront . . . 3
Cart, checkout and orders . . . 3
Account and identity . . . 4
Payments . . . 4
Reviews and ratings . . . 4
Discount codes . . . 5
Marketplace and shops . . . 5
Vendor commission and payouts . . . 5
Returns and refunds (RMA) . . . 6
Admin audit trail . . . 7
Inventory . . . 7
Fulfilment and shipping . . . 7
Product images . . . 8
Bulk catalogue import . . . 8
Tax classes . . . 8
Marketing analytics . . . 8
Exports . . . 9
Email . . . 9
Logging . . . 9
Rate limiting . . . 9
Localisation . . . 10
Theming, branding and storefront templates . . . 10
Catalogue providers . . . 10
Admin back office (/Admin) . . . 11
Vendor area (/Vendor) . . . 11
Cross-cutting guarantees . . . 12
Testing . . . 12
Where it lives . . . 13

A complete list of what the application does, grouped by area and checked against the code in `Domain/`, `Data/`, `Services/`, `Controllers/`, `Areas/`, `Views/` and `Tests/`.

For how to run it, the configuration reference and the reasoning behind the design, see [`../README.md`](../README.md). For how the platform got here — and where it deliberately stops — see [`ECOMMERCE_PLAN.md`](ECOMMERCE_PLAN.md).

Contents

- [Platform and architecture](#platform-and-architecture)
- [Storefront](#storefront)
- [Cart, checkout and orders](#cart-checkout-and-orders)
- [Account and identity](#account-and-identity)
- [Payments](#payments)
- [Reviews and ratings](#reviews-and-ratings)
- [Discount codes](#discount-codes)
- [Marketplace and shops](#marketplace-and-shops)
- [Vendor commission and payouts](#vendor-commission-and-payouts)
- [Returns and refunds (RMA)](#returns-and-refunds-rma)
- [Admin audit trail](#admin-audit-trail)
- [Inventory](#inventory)
- [Fulfilment and shipping](#fulfilment-and-shipping)
- [Product images](#product-images)
- [Bulk catalogue import](#bulk-catalogue-import)
- [Tax classes](#tax-classes)
- [Marketing analytics](#marketing-analytics)
- [Exports](#exports)
- [Email](#email)
- [Logging](#logging)
- [Rate limiting](#rate-limiting)
- [Localisation](#localisation)
- [Theming, branding and storefront templates](#theming-branding-and-storefront-templates)
- [Catalogue providers](#catalogue-providers)
- [Admin back office](#admin-back-office-admin)
- [Vendor area](#vendor-area-vendor)
- [Cross-cutting guarantees](#cross-cutting-guarantees)
- [Testing](#testing)
- [Where it lives](#where-it-lives)

Platform and architecture

- **ASP.NET Core 9 MVC** over the stock template: controllers, Razor views, view components, two areas (Admin, Vendor) and a default route.
- **Two database providers, one model.** `Database:Provider` selects SQLite (default) or SQL Server; nothing below the `DbContext` registration is provider-specific.
- **One-time SQLite → SQL Server import** at startup, through the same EF model, preserving primary

keys (with `IDENTITY_INSERT`) inside a single transaction; it self-skips once the target holds data, so the SQLite file remains a working fallback.

- **Schema creation with** `EnsureCreatedAsync` plus **guarded create-on-demand DDL** for tables that postdate a store (`Log`, reviews, email outbox, coupons, payments, vendor ledger/payouts). Migrations are impossible here (the EF Core Design package is not cached), so this is the documented stop-gap.
- **Seeder** — the demo catalogue, categories, shops, demo accounts and sample coupons, created on first run.
- **Application services behind interfaces** (`ICatalogService`, `ICartService`, `IOrderService`, `IAccountService`, `IInventoryService`, `IShopService`, `IFulfilmentService`, `IReviewService`, `ICouponService`, `IVendorLedgerService`, `IPayoutService`, `IImageStore`, `IEmailTransport`, `IPaymentGateway`/`IHostedPaymentGateway`, `IShippingCarrier`, `IThemeCatalog`, `ISlugService`, `IPasswordHasher`), so each concern can be replaced by a single registration.
- **Session state**, **global antiforgery validation**, and **area authorization policies** (`AdminOnly`, `VendorOnly`, `CustomerOnly`).
- **Provider-specific workarounds are documented in code** — decimal aggregation in memory, client-side rowversion, `EF.Constant` for captured-collection `Contains`, and clamped paging offsets.

Storefront

- **Home** — hero, department tiles, featured products, best sellers, and an order-tracking entry.
- **Catalogue** — browse by department (including sub-categories); free-text search across product name, description, brand and SKU; price-range, brand, in-stock and minimum-rating filters; eight sort orders (including top-rated, most-reviewed and relevance); paging. Every filter is in the query string, so any result set is bookmarkable; soft-deleted and unpublished products never appear.
- **Search suggestions** — type-ahead products and departments from `GET /Catalog/Suggest` with keyboard navigation, degrading to an ordinary search box without JavaScript.
- **Product detail** — image gallery with thumbnail switching, variant selection, live stock messaging ("only N left"), sale pricing with a strike-through, add-to-cart, wishlist, related products, and the full reviews panel.
- **Shop directory and shop storefronts** — every shop listed, and a themed storefront per shop with a hero, a filter rail and the product grid.
- **Order tracking** — guests track an order by order number and email.
- **Informational pages** — privacy, not-found and error (rendered through status-code re-execution).

Cart, checkout and orders

- **Cart** — line editing, per-line stock warnings, a free-shipping progress hint, discount codes (apply/remove, re-priced on every render), live totals, an empty state, and a JSON summary endpoint (`GET /Cart/Summary`).
- **Checkout** — a validated contact and shipping form, a saved-address picker, optional save to the address book, the payment choice (cash on delivery, or card when a hosted gateway is configured), an order summary, and a confirmation page with a status timeline.
- **Idempotent checkout** — a per-form key issued into session must match on POST, and

`Order.IdempotencyKey` is uniquely indexed.

- **Order history and detail** — for signed-in shoppers, alongside guest tracking.
- **Order state machine** — `Pending` → `Paid` → `Shipped` → `Delivered`, plus `Cancelled` and `Refunded`; only legal transitions are allowed. Cancelling restores stock; refunding calls the payment gateway.

Account and identity

- **Register, sign in, sign out**, profile, and password change.
- **Password reset by emailed link** — a signed, expiring token rather than a database row.
- **Address book** — multiple saved addresses with a default.
- **Wishlist** — per-account, toggled from a product page.
- **Two-factor authentication (TOTP)** — optional authenticator-app verification at sign-in, with a set of one-time backup codes for recovery; can be enabled and disabled from the account profile.
- **Shopper-to-shop conversations** — customer/vendor messaging threads tied to an order or product, with read receipts and an inbox for both sides.
- **Two-factor authentication (TOTP)** — optional authenticator-app verification at sign-in, with a set of one-time backup codes for recovery; can be enabled and disabled from the account profile.
- **Custom cookie authentication instead of ASP.NET Core Identity** (the Identity package is absent from the offline NuGet cache): PBKDF2-HMAC-SHA256 with 210,000 iterations and constant-time comparison, a five-attempt lockout with a 15-minute window, a security stamp rotated on credential change, and generic sign-in failure messages.

Payments

- **Cash on delivery** by default.
- **Hosted card flow** behind a swappable pair of interfaces — `IPaymentGateway` (charges a card) and `IHostedPaymentGateway` (redirect to the provider's page plus webhook parsing). The bundled **simulator** is both sides of the exchange; `sandbox` mode is cash-on-delivery only.
- **Signed webhooks** — the raw request body is read and `X-Payment-Signature` verified (HMAC-SHA256) *before* the body is deserialised. An unverified delivery is recorded with `SignatureValid = 0` and answered **401**.
- **Idempotent delivery** — every event is written to `PaymentEvents`, uniquely keyed by `(Provider, EventId)`; a redelivery is answered **200** `duplicate` and changes nothing.
- **Declines don't fail the order** — a declined event marks the attempt `Failed` and leaves the order `Pending`, so the shopper can retry.
- **Unknown orders are acknowledged (200)**, not retried forever; an unknown provider is a genuine 404.
- **Refunds** are routed back through the gateway.

Reviews and ratings

- **Verified-purchase eligibility** — a signed-in shopper with a paid, shipped or delivered order line for the product; one review per shopper per product, editable in place once it exists.
- **Publish immediately, moderate reactively** — `Reviews:AutoPublish`, with a queue for the minority that break the rules.

- **Helpful / not-helpful voting** — one vote each, changeable by voting again; "most helpful" ordering reflects the tally.
- **Seller replies** — the selling shop may post one public reply and edit or clear it.
- **Derived aggregates** — star histograms, per-product averages and shop scores computed at query time; no denormalised rating column exists to drift.
- **Seller reputation badges** — *Star seller* and *Top rated seller* from configurable thresholds, shown in the shop directory, a shop's storefront and the vendor workspace.

Discount codes

- **Percent or fixed-amount** discounts with a minimum subtotal, a validity window, a global `MaxUses`, a per-shopper `MaxUsesPerUser`, and optional restriction to one shop.
- **One code at a time** — applying another replaces it; the session holds the bare code and the cart is re-priced against the current basket on every render.
- **Re-checked at order time** — nothing relies on what the browser last showed; a discount is capped at the goods it applies to, so a total can never go negative.
- **Redemption recorded** (`CouponUsage` keeps the user and the order), which is how a per-shopper cap is enforced without trusting the client; guests are held only to the global limit.
- **Admin CRUD** with validation, an activate toggle (redeemed codes are deactivated, not deleted), and seeded demo codes (`WELCOME10`, `POST5`, `EXPIRED`).

Marketplace and shops

- **Shops** with an approval status, owners and shop users/roles, and contact and branding data.
- **Shop listings** — per-shop price and stock layered over a shared product and variant catalogue, carrying a concurrency token.
- **Vendor onboarding** — a vendor registers a shop and reaches a workspace.
- **Shop themes** — six storefront templates applied by configuration from `ThemeCatalog`, chosen at `/Vendor/Appearance` with live previews (see [Theming, branding and storefront templates](#theming-branding-and-storefront-templates)).

Vendor commission and payouts

- **Append-only ledger** (`VendorLedgerEntry`) of sales and reversals — the only earning record.
- **Credited when money is actually collected** — card capture, cash-on-delivery delivery, or staff marking an order paid — through one shared writer, and reversed on cancellation or refund.
- **Exactly once** — every write is idempotent per order and type and commits as it writes, so a redelivered webhook or a double-clicked button cannot double-credit.
- **Commission withheld on goods only**, rounded half-up (`Math.Round`, `AwayFromZero`); the rate is clamped, and shopless orders are skipped.
- **Per-category commission rates** - a category can override the shop's default rate; the effective rate resolves to the most specific (category) rate when present.

- **Derived balance** — what a shop is owed is always the sum of its unsettled ledger entries, never a stored number.
- **Settlements** (`VendorPayout`, reference `PAY-{year}-{n}`) stamp the entries they cover with a timestamp and reference rather than writing earnings, so a statement is reproducible from the ledger alone.
- **Settlement outcomes** — a payout is `Paid`, `Failed` or `Reversed`. The bank refusing a transfer does not edit any figure: **Mark failed** records the reason, stamps `FailedUtc`, and releases every entry it covered back to the unpaid balance, so the shop can be settled again immediately. **Reversing** (recorded in error) releases the entries the same way while the payout row keeps its totals for the statement — the money trail never rewrites a financial document. Both actions are admin-only, guard against double-transitions, and are audited.
- **Payout destination** — each shop records where settlements are sent (account name, bank, IBAN), collected at registration and editable by the vendor on their payouts page. The admin shop list shows it at a glance and links to the shop's full history in the audit trail.
- **Admin settlement** at `/Admin/Payouts` with a CSV/Excel statement and exportable history; **vendor view** at `/Vendor/Payouts`, read-only for money (only the bank details are theirs to edit), with status badges and the failure/reversal reasons shown on their own settlements.
- **Shop notified** — the shop's contact email gets a `payout-issued` message per settlement (skipped, by design, when the shop has no email on file).
- `Payouts:MinimumAmount` sets the threshold below which a balance is not worth a transfer; a refund arriving after settlement leaves the shop owing the marketplace until new sales cover it.

Returns and refunds (RMA)

- **Full return lifecycle** — a shopper raises a return on a paid/shipped/delivered order (`/Orders/RequestReturn/{id}`, one open request per order), the shop **approves or rejects** it from `/Vendor/Returns`, the goods come back and are **received and restocked**, and the administration **issues the refund** at `/Admin/Returns`. Money moves only in that last step; the customer can withdraw an open request.
- **References** — human-facing `RMA-{year}-{n}`, visible to both the shopper and the shop.
- **Per-line, per-unit scope** — a return names order lines and unit counts, never vague amounts. The request form shows each line's number of units still returnable (everything ordered minus the units past returns already refunded), the server re-enforces the same arithmetic, and the order carries a running `RefundedTotal`.
- **The refund math** — returned goods (with any order-level discount apportioned by value) plus the proportional share of tax. Shipping is refunded only when the whole order comes back. The total is capped both by the ledger amount still owed for the sale and by `GrandTotal − RefundedTotal`, so no return can ever pay out more than the order received. Demo arithmetic only — no gateway always honours a partial, real acquirers may differ.
- **Partial ledger reversal** — a refund reverses only the refunded goods from the shop's sale entry (commission reversed with it), idempotent per `(order, type, return)`, so a double-click refunds the ledger once. An order whose sale was never credited (pre-ledger data) refuses the refund loudly rather than paying without reversing.
- **Restock with a pool preference** — received goods return to the shop's `ShopListing` stock pool when one exists, falling back to the shared variant stock otherwise; every restock writes a return-stamped stock movement.

- **Status machine, not booleans** — `Requested` → `Approved` → `Received` → `Refunded`, with `Rejected` and the shopper's `Cancelled` as exits; transitions are validated in the service layer exactly like order states.
- **Customer visibility** — the order page lists the order's returns with status badges and lets the shopper cancel an open request; the vendor's order detail shows the same returns from the shop side.
- **Admin queue and reports** — `/Admin/Returns` with status/search filters, open-workload counters (requested/approved/received), CSV and Excel export (200 rows per page view, 500 per export).
- **Who does what** — shops decide on their own returns and see other shops' returns as `404`; the administrator can decide or receive on a shop's behalf when staffed covers are needed, and is the only role that issues the money.
- **Admin order view** — every admin order page shows *Refunded so far* and *Still owed to shopper*, so a partial refund is never a surprise.

Admin audit trail

- **Append-only record of administrative actions** (`AuditLog`) — who did what, to which record, and when, with a short human-readable summary per action.
- **Deliberate entries, not a diff** — each writer records the action it performed (`shop.approved / rejected / suspended / reinstated`, `order.status / order.status.rejected`, `refund.issued / refund.failed`, `return.approved / rejected / received`, `payout.created / failed / reversed`). Failed attempts are recorded too — a trail that hides the failures cannot explain the retries.
- **Loose references** — the actor is stored as the display identity and the subject as `(entity type, entity id)`, so deleting an account or a row never rewrites history.
- **Best-effort by design** — a failing audit write is logged and swallowed rather than allowed to undo already-committed work.
- **The trail page** — `/Admin/Audit` filterable by action, entity, id and actor, with dropdown suggestions derived from the data itself; per-entity history links from the shop pages; CSV and Excel export of the filtered view (200 rows paginated, 500 per export).

Inventory

- **Stock levels and manual adjustments** at `/Admin/Inventory`.
- **Append-only stock-movement ledger** — every change is explainable.
- **Optimistic concurrency** — `RowVersion` on the listing is a concurrency token, so concurrent checkouts raise a conflict instead of overselling, and `OrderService` retries with a freshly loaded cart and fails with a customer-facing message.
- **Restock on cancellation or returned goods** (not on order-level refunds, which leave inventory alone) — a cancelled order's stock returns to the shop's listing pool with a variant fallback, and a received return takes the same route (see [Returns and refunds (RMA)](#returns-and-refunds-rma)).
- **Low-stock thresholds** and inventory-value reporting on the dashboard.

Fulfilment and shipping

- **Shipments** (`OrderShipment`) with a **carrier registry** (`ICarrierRegistry`) and a bundled `SimulatorShippingCarrier` alongside a LibanPost carrier.

- **Shipment events** (`ShipmentEvent`) that derive the order's status, never moving it backwards and never resurrecting a cancelled order.
- **Cash-on-delivery collection status** tracked from awaiting collection to collected at delivery.

Product images

- **Uploads behind `IImageStore` / `LocalImageStore`**, served as static files from a configurable request path, so an image is a URL like any other.
- **Bytes checked, not just names** — allowed extension *and* sniffed magic bytes must agree, with a size bound and a per-product cap.
- **Named by the store** — never by the client — with a bounded read.
- **Merged with typed URLs**, so a gallery can mix uploads and links; a rejected upload fails the whole save and rolls back files already written, and removing a URL deletes its file when no other product references it.

Bulk catalogue import

- **CSV import into a chosen shop** at `/Admin/Import`, with a downloadable template.
- **All-or-nothing** — every row is validated before anything is written, so a bulk load cannot leave a half-successful catalogue; failures are reported by line number.
- **Column order independent** — headers are matched by name; only `Name`, `Price` and `SKU` are required. Matching is by SKU, then slug, then name.
- **The exact inverse of the export writer** — a hand-written RFC 4180 reader (quoted fields, doubled quotes, embedded newlines, CRLF/LF and a UTF-8 BOM) round-trips a file the store exported, verified by test.
- **Gallery-safe** — omitting the `Images` column leaves an existing gallery untouched; supplying it replaces it. An update-only mode refuses to create products that do not exist.
- **Locale-independent** — prices parse with a decimal point regardless of the machine's culture.

Tax classes

- **Named rates** (`TaxClass`) that a product may reference; a product with no class falls back to `Store:TaxRate`, so existing orders are unchanged by the feature's arrival.
- **Per-line taxation** — each line is taxed at its own class's rate, with an order-level discount apportioned across lines by value, so a mixed basket is taxed correctly.
- **Exemption** — a class can be permanently zero whatever its rate says.
- **Guarded deletion** — a class still referenced by products is deactivated, never deleted, because deleting it would silently move those products onto the store rate.
- **One calculation, two callers** — `ITaxCalculator` is used by both checkout and the cart preview, so what a shopper is shown cannot disagree with what the order records.
- **Admin CRUD** at `/Admin/Tax` with a CSV/Excel export; rates are validated as fractions (0–1).

Marketing analytics

- **Dashboard** at `/Admin/Marketing` — revenue over time, top products by revenue and by units,

revenue by category, best customers, payment-method mix, coupon performance and fast-moving stock.

- **Eight exportable sections** as CSV or Excel: `daily-revenue`, `top-products-revenue`, `top-products-units`, `revenue-by-category`, `top-customers`, `payment-methods`, `coupons`, `fast-moving`.
- **Collected money only** — pending, cancelled and refunded orders are not revenue.
- **Coupon performance reads the discount from the order**, never reconstructing it from the coupon's own rules.
- **Built on the same figures the vendor dashboard uses**, so the marketplace and a shop cannot disagree about a shared number.

Exports

- **CSV and Excel** from a single description (`TabularData` → `ExportBuilder`), honouring the filters currently applied, so the file and the page cannot disagree.
- **CSV that opens correctly** — UTF-8 with a byte-order mark, invariant-culture numbers and dates, and formula-injection neutralisation for cells beginning `=`, `+`, `-` or `@`.
- **A real workbook** — numbers Excel can sum, real dates, a bold frozen header and an autofilter.
- **Bounded** by the page's cap (orders 200, products 5,000).
- **Payout statements** and settlement-history exports beyond the standard order/product/customer reports.

Email

- **Transactional outbox** (`EmailMessages`) — nothing sends on the request path; a request queues a row and a background dispatcher delivers it.
- **Idempotent notifications** via `DedupeKey`, so a retried request cannot send twice.
- **Retry with backoff**, failure parking with the last error, and **one-click resend**.
- **Templates** for order confirmation, shipping, cancellation, refunds (order-level and per-return), return decisions, payout settlements, shop onboarding decisions, password reset and welcome — as both HTML and text. A payout or shop-decision message reuses the dedupe rule: the same fact (per payout, or per shop+decision+reason) never mails twice.
- **Transports** — `File` (writes a genuine `.eml` and an `.html` copy, no mail server needed) or `Smtp`; `Email:Enabled = false` records messages as *Skipped* rather than dropping them.
- **Admin outbox page** — filterable by state and type, with a rendered preview.

Logging

- **Console, debug and a database sink** — every line is also written to the store's `Log` table.
- **Non-blocking and self-suppressing** — callers enqueue; a background service batches writes, and the sink never logs its own SQL.
- **Guaranteed table** via `LogTable.EnsureAsync`, an independent level filter, and optional retention.

Rate limiting

- **Fixed-window limiter** from the shared framework (no package) with four named policies — `auth`, `checkout`, `review` and `webhook`.

- **Partitioned** per signed-in user, or per IP address for guests, so one shopper cannot exhaust another's allowance.
- **429 with** `Retry-After` on rejection, logged, and switchable with `RateLimiting:Enabled = false`.

The webhook policy is deliberately generous, because a provider redelivering after an outage must not be throttled into giving up.

Localisation

- **Three languages** — English (default), Arabic (right-to-left) and French — offered by a language switcher in the storefront header. Catalogs are plain `key=value` files in `App_Data/i18n/`, so adding a language is a file and one configuration entry.
- **Precedence** — the shopper's explicit choice (a cookie), then their browser's `Accept-Language`, then the store default. Only languages the store actually offers are honoured.
- **Graceful fallback** — a missing translation degrades to English and then to the key, so a gap in a catalogue never renders a blank page or fails a request.
- **RTL support** — `dir="rtl"` on `<html>` plus explicit mirroring in `wwwroot/css/i18n.css`; prices, order numbers and codes are kept left-to-right inside Arabic text so numbers cannot reverse.
- **Scope, stated plainly** — the shared chrome (header, navigation, footer, cart and checkout labels) is translated. Product content and the admin area remain English-only; see *Known limitations*.

Theming, branding and storefront templates

- **Brand palette emitted as CSS custom properties from configuration** — colours change in `appsettings.json` and nowhere else; `wwwroot/css/brand.css` is the single stylesheet.
- **Brand conventions** reproduced deliberately (yellow buttons with an outline-on-hover, blue links that turn yellow, tight corners).
- **Six storefront templates** — `postal-classic` (the default), `midnight`, `sunrise`, `boutique`, `forest` and `mono` — resolved through `IThemeCatalog`, which always falls back to the default so a blank or unknown `Shop.ThemeKey` can never render an unstyled storefront.
- **A storefront-wide theme** — `Store:Theme` (`1` = Amazon-style, `2` = eBay-style, `3` = Etsy-style, `4` = *Midnight & Gold*, `5` = *Poshmark-style*) emits `data-store-theme` on `<html>`; `wwwroot/css/themes.css` overrides the Bootstrap and brand custom properties, re-skinning the whole storefront — including the dashboards, which win through Bootstrap variable overrides.
- **Live previews** — each picker card carries its own theme variables inline.
- **Dual currency display** — USD prices shown with an indicative LBP figure, using the configured exchange rate.
- **Dual currency display** — USD prices shown with an indicative LBP figure, using the configured exchange rate; toggleable per shop.

Catalogue providers

- **Local (default)** — EF Core over the configured store.
- **Shopify, mocked** — real Shopify response shapes from a local fixture, no token and no network.

- **Shopify, live** — the Storefront API over GraphQL, with a search-syntax translator, cursor-walking paging bounded by a cost limit, and caching of brands, price bounds and collections.
- **Three mismatches handled explicitly** — cursor vs page-number paging, client-side price filtering (with an estimated count), and global-ID parsing into `int` keys with a deterministic fallback.

Admin back office (`/Admin`)

- **Dashboard** — revenue (lifetime and 30-day), order counts by state, inventory value, stock alerts.
- **Products** — search/category/state filters (active, draft, out of stock, deleted); create and edit with the full variant matrix and an image gallery; publish toggle; soft delete and restore. Staff see soft-deleted rows too, so a withdrawn product can be found and restored.
- **Import** — bulk CSV product load into a chosen shop, all-or-nothing, with a downloadable template (see [Bulk catalogue import](#bulk-catalogue-import)).
- **Categories** — the full tree, with product counts and cycle guards.
- **Inventory** — stock levels, manual adjustments, and the stock-movement ledger.
- **Orders** — a filterable queue, detail, and legal status transitions (cancelling restores stock, refunding calls the gateway); the detail page carries the order's returns and the refunded/owed figures.
- **Returns** — the RMA queue with status filters and exportable CSV/Excel; the detail page can decide or receive on the shop's behalf and is the only place money is issued, with the per-return audit history beneath it (see [Returns and refunds (RMA)](#returns-and-refunds-rma)).
- **Customers** — search, detail with lifetime spend, activate/deactivate.
- **Coupons** — create and edit discount codes, with an activate toggle.
- **Tax** — named tax classes with per-line application and guarded deletion (see [Tax classes](#tax-classes)).
- **Shops** — approve, reject, suspend and reinstate sellers, set each shop's commission rate, see
- **Translations** — a database-backed translation catalogue with an admin screen for editing keys.
- **Listing moderation** — a vendor's listing goes live only after admin approval; the admin can approve or reject with a note; rejected listings can be resubmitted. the payout destination recorded with each shop, and jump straight to that shop's audit history.
- **Payouts** — every shop's outstanding balance, a one-click settlement with statements and an exportable history, and the mark-failed / reverse paths that release entries without editing financial rows (see [Vendor commission and payouts](#vendor-commission-and-payouts)).
- **Marketing** — revenue, product, category, customer, payment-method and coupon analytics, all exportable (see [Marketing analytics](#marketing-analytics)).
- **Reviews** — the moderation queue, filterable by state and rating.
- **Audit trail** — the administrative history page with filters and export (see [Admin audit trail](#admin-audit-trail)).
- **Emails** — the outbox, filterable, with preview and resend.
- **Exports** — orders, products and customers as CSV or Excel, honouring the current filters.

Vendor area (`/Vendor`)

- **Dashboard** — product, order, gross-sales and outstanding-payable figures for the vendor's shop,

plus a daily-revenue trend, top products, payment-method mix, stock alerts and recent orders — a scoped view of the same analytics the admin marketing page shows.

- **Products** — catalogue CRUD scoped to the vendor's own shop, including image uploads.
- **Orders** — the shop's own order book, filterable by status and searchable, with order detail.

Another shop's order is a `404`, not a redacted row. Returns touching the order are shown alongside.

- **Returns** — the shop's RMA worklist: open requests first, decide (approve/reject with a note) and

receive the goods back into stock. Other shops' returns are a `404`.

- **Reviews** — every review against the shop's products, with the aggregate score and badge and a

reply box under each.

- **Payouts** — the outstanding balance, every settlement with its statement (including why a payout

ended up `Failed` or `Reversed`), the recent ledger entries, and the payout destination (account name, bank, IBAN) they can update themselves. Money itself is read-only — settlements are the marketplace's action.

- **Appearance** — the storefront-template picker, with live previews.

Cross-cutting guarantees

- **Money is** `decimal`, never floating point.
- **Orders snapshot everything** (product name, variant name, SKU, unit price), so editing a product

cannot rewrite an order's history.

- **Products are soft-deleted**; sold variants are deactivated rather than removed.
- **Stock is committed under optimistic concurrency**, so concurrent checkouts cannot oversell.
- **Checkout is idempotent**, and **every stock change is explainable** through the movement ledger.
- **Antiforgery is global** — a POST without a token returns 400.
- **Ratings and vendor balances are derived, never stored.**
- **A page number cannot break a query** — every paged query clamps to an existing page and computes

its offset in `long`.

Testing

- **xUnit suite (287 tests)** over EF Core's in-memory provider and Moq — no database and no server

needed. It covers the order state machine, stock arithmetic and refunds, the email outbox (dedupe, retry/backoff, skip, resend, counts), the password-reset token lifecycle, paging maths, the email templates, the rate-limit configuration, the coupon rules, the vendor ledger and payouts, and the export writers (CSV escaping, BOM, formula neutralisation, and the Excel round-trip). It also covers the CSV reader and bulk importer (round-trip, validation, all-or-nothing), the tax arithmetic (per-line rates, exemption, discount apportionment and the rounding cent), the marketing dashboard figures (coupon maths, collected revenue only, category totals, and order-line snapshots surviving a deleted variant or a renamed product) and language resolution (cookie → `Accept-Language` → default, with English fallback). The returns suite asserts the refund arithmetic to the cent (goods + tax share, shipping only on a full return, ledger and grand-total caps), the lifecycle transitions and guards (one open return per order, spent units, unpaid orders), the listing/variant restock routes, idempotent partial ledger reversals, payout mark-failed/release and reverse/release, and the registration collecting the payout destination; the notification suite covers the payout and shop-decision emails and their dedupe keys; the audit suite covers recording, filters and the best-effort swallowing of a failing write.

Where it lives

Area	Location
Domain entities, enums and configuration models	`Domain/`
DbContext, seeder, guarded DDL, SQLite→SQL Server import	`Data/`
Application services and their settings	`Services/`
Email outbox, templates, transports, password reset	`Services/Email/`
CSV / Excel writers	`Services/Export/`
Bulk product import	`Services/Imports/`
Tax calculation	`Services/TaxCalculator.cs`
Language resolution and catalogs	`Services/Localization/`, `App_Data/i18n/`
Image storage and validation	`Services/Images/`
Database logging sink	`Services/Logging/`
Shopify catalogue adapter	`Services/Shopify/`
Shipping carriers	`Services/Shipping/`
Storefront controllers	`Controllers/`
Admin back office	`Areas/Admin/`
Vendor workspace	`Areas/Vendor/`
Razor views and view components	`Views/`, `ViewComponents/`
xUnit suite	`Tests/`
Brand stylesheet	`wwwroot/css/brand.css`
Configuration	`appsettings.json`, `appsettings.Development.json`